



南京林业大学
NANJING FORESTRY UNIVERSITY

FeHALS：基于 Web 的 3D 可视化航路规划与激光仿真系统 技术设计书

课程名称： “智能 +”GIS 设计与开发

专业： 测绘工程

组别： 7 组

年级： 2023 级

小组成员：	王孜悠	2310604116
	梅康凯	2310604109
	唐如意	2310604115
	杨杰瑞	2310604122
	温璞	2310604117
	刘继明	2110604112
	李英杰	2310604106
	杜仲宇	2310604103
	施祺	2310604113
	向宇鸣	2310604118

指导老师： 隋铭明 那嘉明 陈动 朱杰

批改日期：

提交日期： 2026 年 9 月 16 日

目录

1 背景目的 2

1.1 项目背景 2

1.2 建设目标 2

1.3 设计原则 3

2 需求分析 3

2.1 业务流程 3

2.2 功能需求 3

2.3 与现有工具的对比分析 3

2.4 非功能需求与约束 4

3 总体设计 4

3.1 系统架构 4

3.2 数据与控制流程 6

3.3 关键设计决策 7

4 功能设计 7

4.1 模型、场景与航点 7

4.2 参数、轨迹和配置 8

4.3 仿真、日志和结果 9

4.4 接口设计 9

5 数据分析 10

5.1 数据对象与组织 10

5.2 坐标、统计与性能 10

5.3 数据类型与 GIS 集成 10

6 主要技术手段 11

6.1 前端与三维显示 11

6.2 后端与 HELIOS++ 11

6.3 协作、测试与改进 12

1. 背景目的

1.1. 项目背景

机载激光扫描 (Airborne Laser Scanning, ALS) 能够直接获取地表及地物的三维离散采样, 是数字高程模型生产、森林调查、城市三维建模和灾害监测的重要数据源。真实飞行采集受设备成本、空域、天气和任务风险制约, 外业前通过虚拟场景试验航线与扫描参数, 可以降低反复试飞的成本。

HELIOS++ 是目前最为活跃的开源激光雷达仿真框架之一, 采用 CPU 光线追踪模拟激光脉冲从发射到返回的全波形响应, 支持地基、车载、无人机和机载平台的扫描配置^[1-2]。其核心架构将场景 (scene)、平台 (platform)、扫描器 (scanner) 和测量任务 (survey) 四个维度解耦, 以 XML 文件定义各组件间的组合关系。平台支持线性路径 (linear path) 和插值运动 (interpolated) 两种运动模型, 其中插值运动模型通过读取时间戳序列的轨迹文件 (.trj), 在相邻航点间线性插值获得平台位置与姿态, 再根据 ARINC 705 规范将偏航角 (yaw/heading) 转换为绕 Z 轴的旋转, 以驱动扫描器产生与航线正交的扫描线。这一设计使 HELIOS++ 能够在不依赖真实飞控数据的情况下, 仅凭航点坐标和偏航角序列即可生成物理上合理的激光扫描模拟。

然而, HELIOS++ 的原生操作方式为命令行和 XML 配置, 用户需手动编写场景、平台、扫描器、survey 等多个 XML 文件, 并管理轨迹文件与资产的路径引用。对于初学者或测绘工程专业的学生而言, 这一过程存在较高的入门门槛: 参数的单位与范围不直观、轨迹与配置的关联关系隐含在 XML 索引中、仿真结果缺乏即时的可视化反馈。此外, HELIOS++ 官方提供的 QGIS 插件 AEOS^[3] 虽在一定程度上降低了操作门槛, 但 AEOS 依赖于 QGIS 桌面环境, 安装配置复杂, 且在使用过程中仍需用户具备一定的 QGIS 操作经验。

FeHALS (Frontend of HELIOS++ for Airborne Laser Scanning) 正是在这一背景下提出的, 它以浏览器为操作入口, 将三维预览、航点编辑、弓字形航线生成、参数配置、HELIOS++ 任务执行、实时日志和点云分析串联为完整流程, 旨在为 HELIOS++ 提供一个现代化、免配置的 Web 前端套件。开放式 WebGIS 技术便于跨平台访问与后续扩展^[4-6], 浏览器侧利用 WebGL 和 Three.js 完成硬件加速三维显示, 无需安装专用桌面 GIS 软件。在 GIS 应用层面, FeHALS 将激光雷达仿真中的坐标系、高程模型、航线规划、点云统计分析等 GIS 核心概念融入 Web 工作流, 使仿真过程与 GIS 工作流自然衔接^[7]。

1.2. 建设目标

本项目旨在为 HELIOS++ 提供直观、可交互、可追踪的前端工作台: 集中管理模型、航迹和扫描参数; 在 Z-up 三维场景中完成航点规划; 生成 HELIOS++ 所需轨迹和 XML; 以异步任务运行仿真并实时显示日志; 解析 XYZ、LAS、LAZ 结果, 提供点云渲染与统计; 形成便于多人协作和迭代的 WebGIS 工程结构。

系统面向激光雷达仿真学习者、测绘工程学生和方案验证人员, 定位为教学与原型验证工具, 不替代实际航空作业中的空域审批、飞控、坐标转换及成果质量验收。本设计书以 GitHub 仓库 master 分支当前实现为准, 规划中的 LOD、任务队列、Docker 等功能不会当作现成功能描述。

1.3. 设计原则

系统遵循坐标与参数一致、操作可视化、任务可追踪、前后端模块化和能力渐进增强原则。前端、后端和 HELIOS++ 均以 Z 轴表示高程；任务具有唯一标识、状态、日志和结果；组件、状态、接口和服务分层；当前先满足课程演示和百万点级预览，超大点云与生产部署作为后续建设内容。

2. 需求分析

2.1. 业务流程

一次完整业务流程为：启动前后端并检查 HELIOS++ 环境；上传场景模型并检查方向和包围盒；手工编辑航点或由矩形生成弓字形航线；设置平台、飞行与扫描参数；生成轨迹和配置；提交仿真并查看实时日志；加载结果点云、查看统计与直方图；下载结果或清理缓存。

在仿真数据源方面，系统可处理两类典型的三维场景数据：一类是建筑群级别的模型集合，如 BuildingWorld 数据集中的单体建筑模型，这类模型细节丰富、结构复杂，适合用于验证仿真参数对不同建筑形态的响应差异；另一类是建筑内部结构模型，如 Building3D 数据集中的精细室内场景，其空间尺度小、遮挡关系复杂，适合用于评估激光穿透率和点云完整性。两类数据集对于验证 FeHALS 的航高自动计算、扫描角度配置和点云覆盖度分析等功能具有重要参考价值。

2.2. 功能需求

系统功能需求如Table 1所示。

表 1: FeHALS 功能需求。

编号	需求说明
FR-01	上传 OBJ、GLTF、GLB、STL；列出、显示/隐藏、删除模型，显示包围盒。仅 OBJ 作为当前 HELIOS++ 场景输入。
FR-02	提供旋转、缩放、平移、视角适配、Z-up 网格和模型方向转换。
FR-03	支持点击添加、拖动、右键或 Delete 删除、列表编辑和清空，并以折线连接。
FR-04	选择矩形范围和航线间距，自动生成连续往返的弓字形航点。
FR-05	以所有模型世界包围盒的最大 Z 加 20m 计算建议高度，并受平台范围约束。
FR-06	设置 UAV/Airborne、速度、高度、扫描频率、扫描半角、脉冲频率和输出格式。
FR-07	生成恒定高度轨迹文件和 HELIOS++ survey XML。
FR-08	异步调用 HELIOS++，记录 pending/running/completed/failed 状态并支持取消。
FR-09	WebSocket 实时推送日志，REST 接口可补取历史日志。
FR-10	查询、下载和解析结果；显示点数、边界、高程/强度统计及 40 组直方图。
FR-11	统计并分类清理 models、trajectories、configs、results。

2.3. 与现有工具的对比分析

FeHALS 不是对现有点云编辑软件或仿真引擎的简单替代，而是定位于它们之间的桥梁：它既不是专业的点云编辑工具如 CloudCompare^[8]，也不是点云处理库如 PDAL^[9]，更不是激光雷达仿真引擎 HELIOS++ 本身，而是将 HELIOS++ 的能力封装为 Web 可访问的图形化工作流。Table 2展示了 FeHALS 与同类工具的功能定位差异。

FeHALS 的核心竞争壁垒在于：将 HELIOS++ 仿真引擎的复杂 XML 配置和命令行操作封装为直观的 Web 界面，同时保留对仿真参数的精细控制权。与 AEOS 相比，FeHALS 不依赖 QGIS 桌面

表 2: FeHALS 与同类工具的功能定位对比。

维度	FeHALS	AEOS (QGIS Plugin)	CloudCompare	PDAL
定位	HELIOS++ Web 前端	HELIOS++ QGIS 插件	点云编辑与可视化	点云处理库
安装方式	浏览器访问	QGIS 插件安装	桌面应用安装	Python/C++ 库
三维场景编辑	交互式 3D 场景	无独立 3D 场景	专业 3D 编辑	无
航迹规划	航点 + 弓字形生成	需手动绘制	不支持	不支持
仿真驱动	一键执行	需切换至 HELIOS++	不支持	不支持
点云分析	全量统计 + 直方图	有限	丰富	丰富

环境，安装配置成本更低；与 CloudCompare 和 PDAL 相比，FeHALS 专注的是仿真前的规划和仿真后的即时分析，而非点云的专业编辑和处理。三者之间是互补关系——用户可以在 FeHALS 中完成仿真规划与初步分析，再将结果导出至 CloudCompare 或 PDAL 进行深度处理。

2.4. 非功能需求与约束

易用性 核心流程在单页完成，参数显示单位、范围和默认值；三维场景支持鼠标拖拽、滚轮缩放等直观操作；参数校验在提交前给出明确提示。

性能 全量统计与显示数据分离，前端渲染最多一百万点，XYZ 每五十万行分块读取；统计计算基于全量点集，确保准确性。

可靠性 子进程失败不拖垮 API，模型异步加载期间删除不产生残留对象；WebSocket 断线后自动重连，日志可通过 REST 补取。

可维护性 视图、Pinia 状态、API 客户端、路由、数据模型和业务服务解耦；后端按 API 路由、Pydantic schema、业务 service 和 config 分层。

系统依赖正确安装的 HELIOS++ 及其环境路径，未配置时只能使用三维预览。UAV 与 Airborne 当前均引用 copter_linearpath 平台和 rieg_l_vux-luav 扫描器，主要由参数范围区分。航点在 $z = 0$ 地面编辑，生成轨迹时统一叠加飞行高度，属于恒高简化模型。仓库说明记录本机构建的 LASlib 输出异常，因此默认 XYZ；LAS/LAZ 只有在 HELIOS++ 正确链接 LASlib 时可用。任务和配置主要存于单进程内存，系统也未设置账户权限，生产部署前必须补充持久化、鉴权、上传限制、HTTPS 和审计。

3. 总体设计

3.1. 系统架构

FeHALS 采用 B/S 三层架构。与传统 C/S 架构的 GIS 系统相比，B/S 架构具有免安装、跨平台、易升级等显著优势——用户无需安装任何桌面软件，仅通过浏览器即可访问整个系统，服务端更新后所有用户立即获得最新功能。这种架构在 WebGIS 领域已得到广泛验证和应用^[5-7]。

表现层基于 Vue 3 组合式 API 构建单页应用（SPA），利用 Vite 实现开发热更新和构建打包^[10]。Vue 3 的 Composition API 允许将逻辑按功能组织为可复用的组合式函数（Composables），例如 useThreeScene 封装所有 Three.js 场景逻辑，useHeliosAPI 封装所有后端 API 调用。Pinia 作为状态管理库^[11]，将导航状态（waypoints store）、场景状态（scene store）和仿真状态（simulation store）

三者分离，避免了组件间跨层级的事件传递。Axios 统一管理 HTTP 请求，支持请求拦截和响应转换。Three.js 负责三维场景的渲染和交互，使用 WebGL 实现硬件加速^[12-13]。

服务层基于 FastAPI 框架^[14]，利用 Python asyncio 实现异步处理。Pydantic 对所有请求和响应进行类型校验，在业务逻辑执行前捕获格式错误^[15]。业务逻辑按服务模块组织：模型管理（model_service）、轨迹生成（trajectory_generator）、配置生成（config_generator）、仿真执行（simulation_runner）和结果解析（result_parser）。WebSocket 端点在任务执行期间推送实时日志，REST 接口用于查询任务状态和结果。

计算层即 HELIOS++ 仿真引擎，通过 asyncio 子进程调用。系统不直接链接 HELIOS++ 的 C++ 库，而是通过命令行参数传递配置，通过标准输出捕获日志，通过文件系统获取结果，这种松耦合方式便于故障隔离和日志记录。

本地目录保存模型、轨迹、配置和结果，模型 manifest 用于服务重启后的恢复。前端只交换 JSON、文件和任务标识，后端负责文件组织、配置生成、进程管理和结果解析。

系统总体架构如图 1 所示。

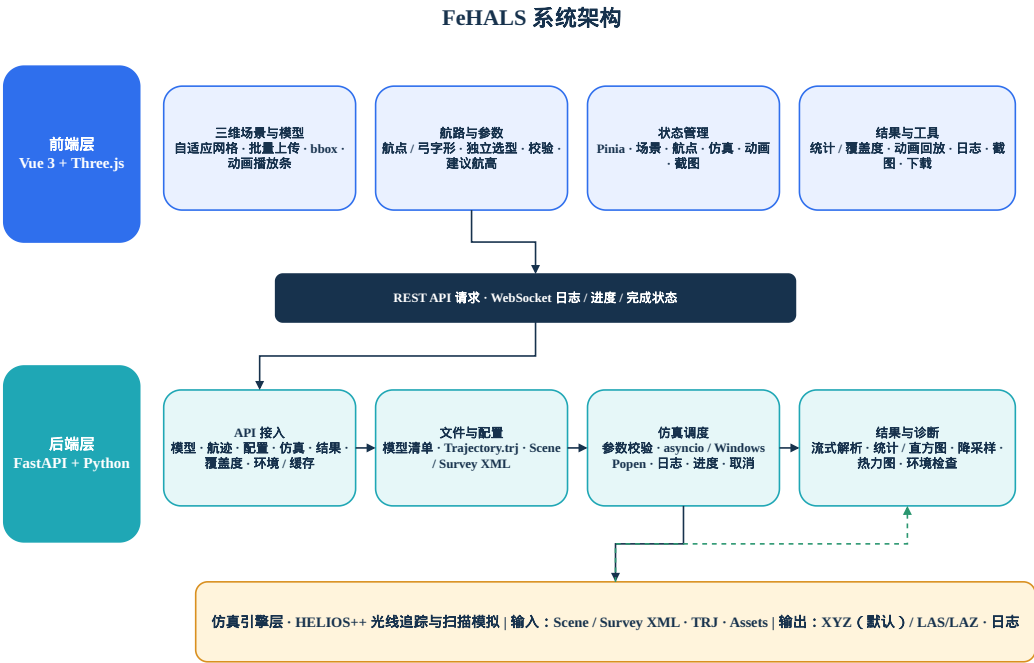


Fig. 1: FeHALS 总体架构

前端中 Scene3D.vue 管理画布，ControlPanel.vue 管理参数，WaypointList.vue 管理航点，Log-Console.vue 展示日志，PointCloudPanel.vue 展示结果。共享状态由 scene、waypoints、simulation 三个 store 管理；跨组件逻辑放在 composables。后端按 API 路由、Pydantic schema、业务 service 和 config 分层。

系统模块架构如图 2 所示。

FeHALS 系统架构图

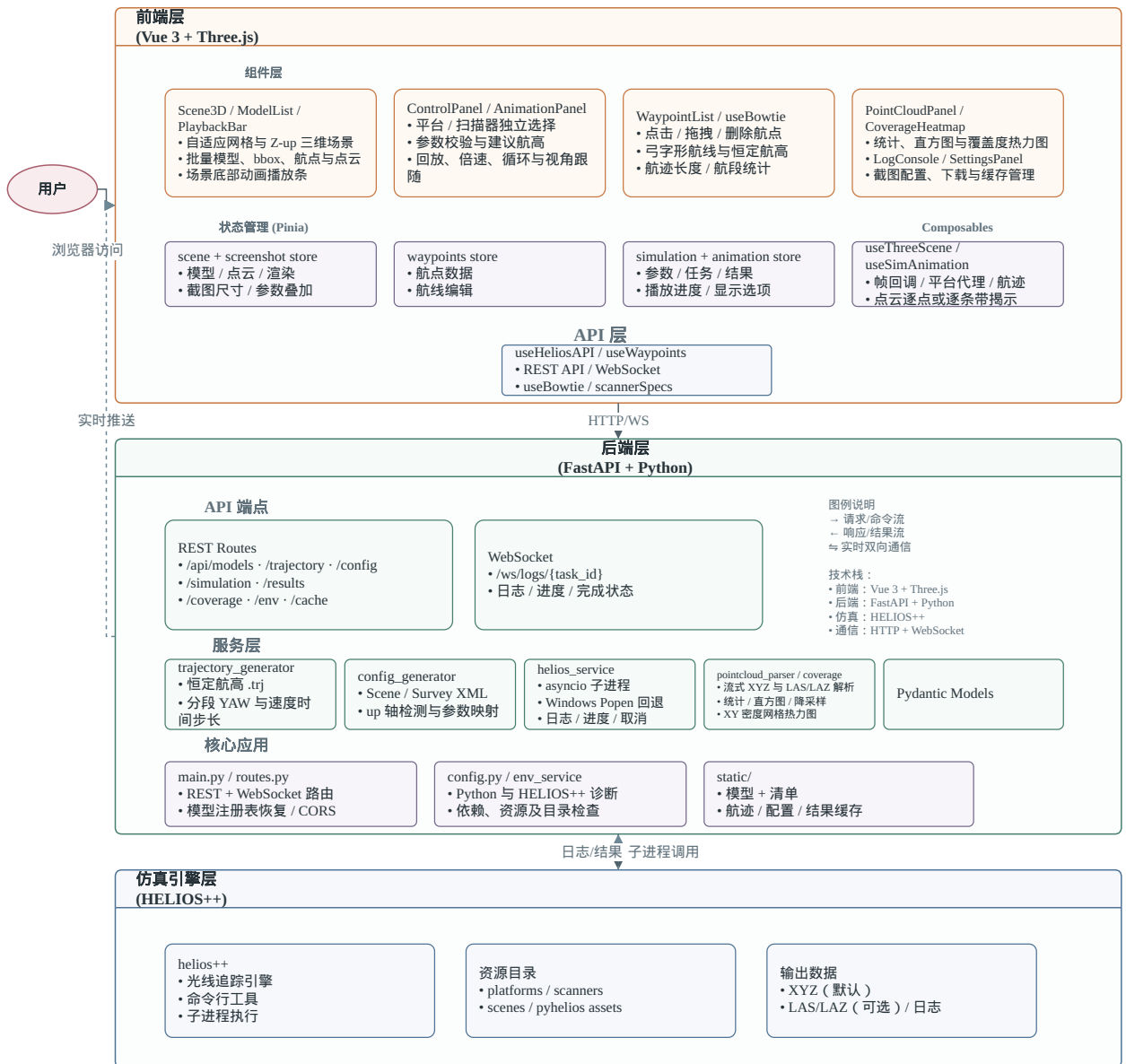


Fig. 2: FeHALS 模块架构

3.2. 数据与控制流程

组件操作先更新 Pinia，再由 useHeliosAPI 请求后端。后端为轨迹、配置和任务分配 UUID，生成文件并启动 HELIOS++ 子进程；日志写入任务缓冲并广播；成功后定位结果文件、计算全量统计并返回显示抽样；前端创建 THREE.Points 绘制点云。

数据流程遵循清晰的单向依赖链：场景模型和航点数据是输入起点，轨迹文件和参数配置由输入数据生成，配置 XML 引用轨迹文件和场景文件，HELIOS++ 读取配置后执行仿真，仿真结果经解析后生成点云统计和渲染数据。这一流程确保了数据变更可以追踪，便于定位问题和迭代优化。

系统工作流程如Fig. 3所示。



Fig. 3: FeHALS 工作流程

开发环境中 Vite 默认使用 5173 端口，并把/api 和/ws 转发到 8000 端口 FastAPI；后端通过/static 发布文件。Windows 用 run.bat，类 Unix 用 run.sh。当前是单机双进程课程演示结构，尚未包括数据库、反向代理、容器编排和集中监控。

3.3. 关键设计决策

采用 Web 前端可以降低分发门槛并把场景、参数与日志放在同一工作区^[5-6]。在技术选型上，Vue 3 被选为前端框架的核心原因在于其组合式 API 天然适合将三维场景、航点编辑、仿真控制等复杂逻辑拆分为独立的组合式函数，每个函数封装特定领域的逻辑和状态，互不干扰。Pinia 相比 Vuex 具有更完善的 TypeScript 类型推断和更简洁的 API，在需要深度追踪仿真任务状态变更时优势明显。Three.js 作为 WebGL 的封装库，提供了一整套三维场景管理工具，无需从零编写着色器即可实现模型加载、航点标记、点云渲染等核心功能。

系统显式将相机 up 设为 (0,0,1)，Y-up 模型加载时绕 X 轴旋转 90°，保证测绘高程语义一致。资源操作采用 REST，连续日志采用 WebSocket^[16-17]。GLTF/GLB/STL 只承担当下前端预览，避免与 HELIOS++ 输入能力混淆。点云采取“全量统计、抽样渲染”，在统计准确性与交互性能间折中；更大数据仍需要层次 LOD 或渐进渲染^[18-19]。

4. 功能设计

4.1. 模型、场景与航点

后端校验扩展名并以随机标识保存上传文件，返回静态 URL。前端按扩展名选择 OBJLoader、GLTFLoader 或 STLLoader，将对象加入 modelsGroup。模型可见性和包围盒可切换，Box3 计算尺

寸。若模型在异步加载完成前已被删除，pendingRemoval 会使迟到结果被释放，避免“幽灵对象”。相机按包围盒自适应，地面为 XY 平面，Z 为高程。

Raycaster 与 $z = 0$ 隐形地面求交：点击添加航点，拖动修改 XY，右键或 Delete 删除，红色球体和折线表示路径，选中航点变黄。拖动时临时禁用相机控制，以 25 像素平方阈值区分点击和拖动。弓字形航线在矩形内按间距 d 生成，条数为

$$N = \left\lceil \frac{y_{\max} * y_{\min}}{d} \right\rceil + 1, \quad (1)$$

相邻扫描线的 X 端点次序交替，形成连续往返路径。

4.2. 参数、轨迹和配置

建议高度取场景包围盒并集最高点 $z_{\max}$ 加 20 m：

$$h_{\text{suggest}} = z_{\max} + 20 \text{ m}. \quad (2)$$

UAV 速度为 0.5–50 m/s, 高度为 3–500 m; Airborne 速度为 10–200 m/s, 高度为 3–5000 m。RIEGL VUX-1UAV 扫描频率为 10–200 Hz, 扫描半角为 1° – 165° , 脉冲频率为 50–550 kHz。

轨迹文件 (.trj) 是 HELIOS++ 插值运动模型的核心输入，其格式为每行包含七个以逗号分隔的字段：时间 (t)、滚转角 (roll)、俯仰角 (pitch)、偏航角 (yaw)、X 坐标、Y 坐标、Z 坐标。系统在生成轨迹时，偏航角由相邻航点间的方向向量实时计算：

$$\text{yaw} = \arctan \left(\frac{x_{i+1} * x_i}{y_{i+1} * y_i} \right). \quad (3)$$

该公式符合 ARINC 705 规范， $\text{yaw}=0^\circ$ 表示正北方向 (+Y)， $\text{yaw}=90^\circ$ 表示正东方向 (+X)，顺时针方向增加。在航点转角处，系统通过生成同一时间戳、同一位置、不同偏航角的两行记录实现瞬时转向，使每个航段保持恒定偏航角，确保扫描线始终正交于航线方向。飞行速度通过目标速度（默认 10 m/s）和航段距离计算时间步长，各航段时间连续累积。

后端将航点改写为统一高度的轨迹文件，并把平台、扫描器、轨迹、速度、频率、半角和格式写入 survey XML。轨迹生成与配置生成分离，便于复用和排错。扫描条带宽度由飞行高度和扫描角决定：

$$W = 2 \cdot h \cdot \tan \left(\frac{\theta}{2} \right), \quad (4)$$

其中 h 为飞行高度， θ 为总扫描角。扫描线间距由飞行速度和扫描频率决定：

$$S = \frac{v}{f_{\text{scan}}}, \quad (5)$$

其中 v 为飞行速度， f_{scan} 为扫描频率。合理的参数配置应使扫描线间距小于或等于激光脚点间距，以保证点云覆盖的完整性。

4.3. 仿真、日志和结果

任务提交后检查 trajectory、config 和可选 scene model，分配 UUID 并由 pending 进入 running。HELIOS++ 标准输出和错误输出逐行保存并广播到/ws/logs/{task_id}；退出码为 0 且找到结果时进入 completed，否则进入 failed。运行中任务可取消。前端保存 taskId、状态、进度、日志和结果，WebSocket 不可用时可通过 REST 查询。

系统还提供 HELIOS++ 运行环境诊断功能，在设置面板中集中展示检测状态。诊断项包括：HELIOS++ 可执行文件的存在性和可执行权限、HELIOS++ 资源目录的完整性（平台定义、扫描器定义、场景资产搜索路径）、静态工作目录的就绪状态。诊断结果以状态徽章和明细列表的形式展示，缺失项给出引导提示，帮助用户快速定位环境配置问题。

LAS/LAZ 由 laspy 读取^[20]，XYZ 分块读取并过滤非法记录。统计包括总点数、XYZ 边界、高程均值、标准差、最小/最大、中位数、P5/P95、40 组高程直方图，以及存在强度字段时的强度统计。渲染超过一百万点时使用覆盖全文件序列的等间隔索引抽样。前端以 BufferGeometry 和 PointsMaterial 绘制，支持固定色、高度色和强度色；归一化为

$$t = \frac{v * v_{min}}{\max(v_{max} * v_{min}, 1)},$$

(6)

再映射至蓝—青—绿—黄—红色带。抽样仅影响显示，不改变全量统计。

4.4. 接口设计

系统后端 REST 接口如Table 3所示。

表 3: FeHALS 后端接口。

方法	路径	说明
POST	/api/models/upload	上传模型
GET	/api/models	查询模型列表
DELETE	/api/models/{model_id}	删除模型
POST	/api/trajectory/generate	生成航迹
GET	/api/trajectories	查询航迹列表
GET	/api/trajectories/download/{file_id}	下载航迹
POST	/api/config/generate	生成配置
POST	/api/simulation/run	创建仿真任务
GET	/api/simulation/status/{task_id}	查询任务状态
GET	/api/simulation/logs/{task_id}	获取任务日志
POST	/api/simulation/cancel/{task_id}	取消任务
GET	/api/results/{task_id}	获取结果统计和抽样
GET	/api/results/{task_id}/download	下载结果
GET	/api/cache	查看缓存统计
DELETE	/api/cache/{cache_type}	分类清理缓存
GET	/api/env/diagnose	环境诊断

主要数据对象如Table 4所示。

表 4: 主要数据对象。

对象	形式	关键字段	用途
场景模型	OBJ/GLTF/GLB/STL	id, name, url, up, bbox	前端均可预览, 当前只有 OBJ 进入仿真。
航点	JSON	x, y, z	Pinia 中编辑, 生成轨迹前可反复修改。
轨迹	文本	id, altitude, path	由航点生成, 供配置引用。
配置	XML	platform, scanner, speed, angle, frequency	供 HELIOS++ 读取。
任务	内存对象	task_id, status, logs, result	当前随服务进程生命周期存在。
结果	XYZ/LAS/LAZ	X, Y, Z, 可选 intensity	保存原始文件并产生统计和显示抽样。

后端用 Pydantic 进行结构校验, 资源不存在返回 404, 参数或格式错误返回 4xx, 子进程失败保留日志。前端在上传、加载、请求和连接处捕获异常并给出可操作提示。公网部署时不应暴露服务器绝对路径和内部堆栈。

5. 数据分析

5.1. 数据对象与组织

后端请求模型包括 TrajectoryRequest、ConfigRequest、SimulationRunRequest 和 SimulationStatus。前端 scene store 管理模型和点云选项, waypoints store 管理航点, simulation store 管理任务、日志、结果与参数。大型 Three.js 对象由单例 useThreeScene 管理, store 只保存业务镜像, 避免响应式系统深度代理图形对象。

5.2. 坐标、统计与性能

距离单位为米, 速度为 m/s, 扫描频率为 Hz, 脉冲频率为 kHz, 扫描角为半角度数。系统当前不读取 EPSG, 也不进行投影转换, 因此模型和轨迹必须预先处于同一以米为单位的坐标系; 后续应增加 CRS 元数据和 pyproj 转换。

设点数为 n 、高程为 z_i , 则

$$\bar{z} = \frac{1}{n} \sum_{i=1}^n z_i, \quad \sigma_z = \sqrt{\frac{1}{n} \sum_{i=1}^n (z_i - \bar{z})^2}. \tag{7}$$

中位数、P5 和 P95 描述主体分布并降低极端值影响。若 $n > m$ 且 $m = 1,000,000$, 显示索引可取

$$i_k = \left\lfloor \frac{k(n+1)}{m+1} \right\rfloor, \quad k = 0, \dots, m+1. \tag{8}$$

该方法简单确定, 但并非空间均匀采样; 超大点云仍需八叉树、多分辨率或渐进策略^[18-19]。

5.3. 数据类型与 GIS 集成

FeHALS 支持多种三维数据格式的输入与输出, 覆盖从模型导入到仿真结果导出的完整数据链路。在输入方面, 系统支持 OBJ、GLTF、GLB 和 STL 四种三维模型格式。其中 OBJ 格式作为 HELIOS++ 原生支持的场景格式, 可通过 objloader 滤镜直接加载到仿真场景中; GLTF/GLB 格式

利用其高效的二进制传输特性，在前端加载和预览时表现更优；STL 格式则提供了与 CAD 建模软件的数据互通能力。模型上传后，系统自动检测模型的 up 轴方向（Y-up 或 Z-up），并在 Three.js 场景中统一转换为 Z-up 坐标系，与 HELIOS++ 的坐标系保持一致。

在输出方面，系统支持 XYZ、LAS 和 LAZ 三种点云格式。XYZ 格式为纯文本格式，每行记录一个点的三维坐标（X、Y、Z）和可选的强度值，兼容性好但占用空间较大；LAS/LAZ 格式为 ASPRS 标准点云格式，支持存储全波形信息和分类标签，但依赖于 HELIOS++ 构建时是否正确链接了 LASlib 库。当前默认使用 XYZ 格式，并在解析时按五十万行分块读取，避免大文件整体载入内存。

在 GIS 集成方面，系统当前采用以米为单位的世界坐标系，不涉及投影转换和地理配准，因此输入的模型和导出的点云均需预先处于同一投影坐标系中。这一设计简化了系统的实现复杂度，但限制了其直接处理地理参考数据的能力。后续可通过引入 pyproj 库实现坐标参考系统（CRS）的元数据管理，支持在 WGS84 经纬度和投影坐标系之间进行动态转换，从而实现与 GIS 数据源的无缝对接。

6. 主要技术手段

6.1. 前端与三维显示

前端采用 Vue 3 组合式 API 构建单页应用，Vite 提供开发与构建^[10]，Pinia 管理共享状态^[11]，Axios 统一请求管理。Vue 3 的组合式 API（Composition API）是前端架构的核心，它允许将 Three.js 场景逻辑、航点交互逻辑、仿真控制逻辑分别封装为独立的组合式函数（Composables），每个函数管理自己的响应式状态和生命周期，避免了选项式 API 中 mixins 的命名冲突和逻辑分散问题。以三维场景为例，useThreeScene 组合式函数封装了场景（Scene）、相机（Camera）、渲染器（Renderer）、轨道控制器（OrbitControls）和光线投射器（Raycaster）的初始化与配置，其他组件通过调用该函数获取场景实例，无需关心底层细节。

在三维渲染方面，Three.js 封装了 WebGL 的底层接口，提供了场景图、材质系统、几何体、加载器和后处理等完整工具链^[12]。系统使用 Scene 对象管理所有三维对象，通过 Group 实现模型、航点和点云的分组管理。模型加载使用 OBJLoader、GLTFLoader 和 STLLoader，每种加载器处理后处理逻辑统一。点云渲染使用 BufferGeometry 存储顶点位置和颜色，通过 PointsMaterial 控制点的尺寸、透明度和着色方式，采用单一缓冲区避免逐点对象开销。场景支持固定色、高度色和强度色三种着色模式，色彩映射通过蓝—青—绿—黄—红的渐变色带实现。WebGL 作为无插件硬件加速的浏览器标准，适合 Web 端地形与点云显示^[13,19,7]。

在交互方面，系统通过 Raycaster 实现鼠标拾取，与一个不可见的 $Z=0$ 平面求交得到航点位置。OrbitControls 提供场景的旋转、缩放和平移。航点使用红色球体表示，选中时变为黄色，白色折线连接各航点形成航线。拖拽航点时临时禁用相机控制，以防止两者冲突。

点云渲染经过性能优化：渲染点数上限为一百万点，超出时使用等间隔索引抽样；颜色和尺寸参数通过滑块实时调整，触发 PointsMaterial 的更新而非重新创建几何体；后续可将点云解析放入 WebWorker 线程，并引入八叉树 LOD 或渐进渲染以支持千万级点云^[18]。

6.2. 后端与 HELIOS++

FastAPI 提供异步路由与 OpenAPI 文档^[14]，Pydantic 在业务执行前完成类型校验^[15]，asyncio 子进程避免长任务阻塞 API，WebSocket 负责连续日志^[17]。REST 有利于资源接口解耦^[16]，WebSocket 适合持续消息。

后端架构分为四个层次：路由层（routers）定义 API 端点和 HTTP 方法映射；模式层（schemas）基于 Pydantic 定义请求和响应的数据结构；服务层（services）实现业务逻辑，包括模型管理、轨迹生成、配置生成、仿真执行和结果解析；配置层（config）管理文件路径、参数范围和运行时配置。各层之间的依赖关系清晰，便于单元测试和功能扩展。

HELIOS++ 使用 C++ 实现，基于光线追踪技术模拟激光脉冲在地物表面的散射和返回过程^[1-2]。其核心算法通过 CPU 光线追踪并行计算每个激光脉冲与场景几何的交点，根据表面材质属性计算回波能量，生成全波形或离散回波信号。系统支持多类场景、平台和扫描器，并允许在物理真实性和计算成本之间进行权衡配置。本系统采用命令行子进程集成方式，通过 Python 的 `asyncio.create_subprocess_exec` 在子进程中启动 helios++ 可执行文件，将标准输出和错误输出通过管道逐行读取，写入任务缓冲并广播到 WebSocket。这种方式便于故障隔离，子进程崩溃不会影响 API 服务进程。

轨迹生成是后端服务的关键模块之一，它将用户编辑的航点列表转换为 HELIOS++ 可直接读取的 .trj 文件。生成过程包括：计算各航段的方向向量并转换为 ARINC 705 规范的偏航角；以目标飞行速度（默认 10 m/s）和航段距离计算时间步长；在航点转角处使用重复时间戳实现瞬时转向。配置生成模块将用户参数映射为 HELIOS++ 的 survey XML 和 scene XML，包括平台类型、扫描器型号、脉冲频率、扫描频率、扫描角度等参数，并正确设置轨迹文件的列索引映射。

点云结果解析支持 LAS/LAZ 和 XYZ 两种格式。LAS/LAZ 由 laspy 读取^[20]，支持全字段解析；XYZ 以文本流逐行读取，每五十万行分块处理，避免大文件内存溢出。NumPy 用于统计计算，包括总点数、三维边界、高程均值、标准差、中位数、分位数和 40 组直方图。Web 端 LiDAR 系统应明确处理、存储和分发边界^[6]；开源 Web 架构可将后端处理与浏览器可视分析组成完整流程^[5]。

6.3. 协作、测试与改进

仓库通过 GitHub 分支、Pull Request 和审查协作；README 说明安装，CHANGELOG 记录演进，TODO 区分已实现与计划项。交付前应补充：轨迹、弓字形、参数边界、XML 转义、XYZ 解析和统计单元测试；所有 REST 路由的接口测试；小型 OBJ 场景的 HELIOS++ 集成测试；模型增删、航点拖拽、日志重连、点云着色的前端测试；10 万、100 万及更大点云的耗时、帧率和内存测试。

后续应优先完成 HELIOS++ 环境诊断、统一错误结构与自动测试；再将任务、配置和资源关系持久化并引入任务队列；随后增加 CRS、空间索引、LOD、覆盖热力图和仿真动画；最后完成 Docker、反向代理、鉴权、HTTPS、配额与监控。当前 FeHALS 已打通“可视化规划—配置—仿真—日志—点云分析”的核心链路，适合作为课程设计和 HELIOS++ 入门工作台。

CRediT authorship contribution statement

见项目描述的 Section CRediT authorship contribution statement。

参考文献

- [1] WINIWARTER L, Esmorís Pena A M, WEISER H, et al. Virtual laser scanning with HELIOS++: a novel take on ray tracing-based simulation of topographic full-waveform 3D laser scanning[J/OL]. Remote Sensing of Environment, 2022, 269. <https://www.sciencedirect.com/science/article/pii/S0034425721004922>. DOI: <https://doi.org/10.1016/j.rse.2021.112772>.
- [2] WEISER H, WINIWARTER L, ANDERS K, et al. HELIOS++: a general-purpose lidar simulation framework[J/OL]. SoftwareX, 2022, 19: 101183. DOI: 10.1016/j.softx.2022.101183.
- [3] 3D Geo Research Group, Heidelberg University. AEOS: a QGIS plugin for HELIOS++ LiDAR simulation[EB/OL]. 2022. <https://github.com/3dgeo-heidelberg/aeos>.
- [4] XIA J, HUANG Q, GUI Z, et al. Open GIS[M/OL]//Springer Nature Reference. Springer Nature Switzerland, 2024. DOI: 10.1007/978-3-031-41748-1.
- [5] GUIMARÃES N, PÁDUA L, ADÃO T, et al. VisWebDrone: a web application for UAV photogrammetry based on open-source software[J/OL]. ISPRS International Journal of Geo-Information, 2020, 9(11): 679. DOI: 10.3390/ijgi9110679.
- [6] KULAWIAK M, KULAWIAK M, LUBNIEWSKI Z. Integration, processing and dissemination of LiDAR data in a 3D web-GIS[J/OL]. ISPRS International Journal of Geo-Information, 2019, 8(3): 144. DOI: 10.3390/ijgi8030144.
- [7] EVANGELIDIS K, et al. 3D point cloud visualization on the web: a review[J/OL]. Remote Sensing, 2021, 13(24): 5110. DOI: 10.3390/rs13245110.
- [8] GIRARDEAU-MONTAUT D. CloudCompare: 3D point cloud and mesh processing software[EB/OL]. 2011. <https://www.cloudcompare.org/>.
- [9] PDAL Contributors. PDAL: point data abstraction library[EB/OL]. 2015. <https://pdal.io/>.
- [10] YOU E. Vite: next generation frontend tooling[EB/OL]. 2021. <https://vitejs.dev/>.
- [11] MOROTE E S M. Pinia: the intuitive store for vue.js[EB/OL]. 2021. <https://pinia.vuejs.org/>.
- [12] CABELLO R. Three.js: JavaScript 3D library[EB/OL]. 2010. <https://threejs.org/>.
- [13] Khronos Group. WebGL specification[EB/OL]. 2011. <https://www.khronos.org/registry/webgl/specs/latest/>.
- [14] RAMÍREZ S. FastAPI[EB/OL]. 2018. <https://fastapi.tiangolo.com/>.
- [15] COLVIN S. Pydantic: data validation using python type hints[EB/OL]. 2019. <https://docs.pydantic.dev/>.
- [16] FIELDING R T. Architectural styles and the design of network-based software architectures[D/OL]. University of California, Irvine, 2000. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [17] FETTE I, MELNIKOV A. The WebSocket protocol: 6455[R/OL]. RFC Editor, 2011. DOI: 10.17487/RFC6455.
- [18] SCHÜTZ M, KRONLACHNER M, WIMMER M. Progressive real-time rendering of one billion points[J/OL]. Computer Graphics Forum, 2020, 39(2): 383-394. DOI: 10.1111/cgf.13938.
- [19] KORDEZ J, MAROLT M, BOHAK C. Real-time interpolated rendering of terrain point cloud data[J/OL]. Sensors, 2023, 23(1): 72. DOI: 10.3390/s23010072.
- [20] MONTAIGU T. laspy: python library for lidar LAS/LAZ IO[EB/OL]. 2017. <https://laspy.readthedocs.io/>.